

Oracle SQL Subquery Logic Review & Execution Analysis

Query Review 1

Original Query

```
SELECT first_name, salary
FROM employees
WHERE salary > AVG(salary);
```

Analysis:

The query is invalid because aggregate functions cannot be used directly inside the WHERE clause. Oracle evaluates the WHERE clause before calculating aggregate functions such as AVG().

Error:

```
ORA-00934: group function is not allowed here
```

Corrected Query

```
SELECT first_name, salary
FROM employees
WHERE salary >
(
SELECT AVG(salary)
FROM employees
);
```

Subquery Type: Scalar Subquery

Business Logic Review

The corrected query properly returns employees earning more than the company average salary.

Query Review 2

Original Query

```
SELECT department_name
FROM departments
WHERE department_id =
(
SELECT department_id
```

```
FROM employees
);
```

Analysis:

The query assumes the subquery returns only one department.

In reality, multiple employees belong to multiple departments, so the subquery returns multiple rows.

Error:

ORA-01427: single-row subquery returns more than one row

Corrected Query

```
SELECT department_name
FROM departments
WHERE department_id IN
(
  SELECT department_id
  FROM employees
  WHERE department_id IS NOT NULL
);
```

Subquery Type: Multi-Row Subquery

Business Logic Review

The revised query correctly returns departments that currently contain employees.

Query Review 3

Original Query

```
SELECT *
FROM employees
WHERE department_id IN
(
SELECT salary
FROM employees
);
```

Analysis

The query is logically incorrect.

Department_ID identifies departments, while Salary represents employee compensation.

These columns have different business meanings and should never be compared.

The query may execute, but the result has no business value.

Corrected Query

```
SELECT *
FROM employees
WHERE department_id IN
(
SELECT department_id
FROM departments
);
```

Subquery Type: Multi-Row Subquery

Business Logic Review

The corrected query follows the relationship between EMPLOYEES and DEPARTMENTS and returns meaningful results.

Query Review 4

Original Query

```
SELECT first_name
FROM employees
WHERE salary >
(
  SELECT salary
  FROM employees
  WHERE department_id = 90
);
```

Analysis:

The query assumes Department 90 contains only one employee.

In the HR schema, Department 90 usually contains multiple employees.

Therefore the subquery returns multiple salary values.

Error:

ORA-01427: single-row subquery returns more than one row

Corrected Query (Option 1)

Employees earning more than the average salary of Department 90:

```
SELECT first_name, salary
FROM employees
WHERE salary >
(
  SELECT AVG(salary)
  FROM employees
  WHERE department_id = 90
);
```

Corrected Query (Option 2)

Employees earning more than every employee in Department 90:

```
SELECT first_name, salary
FROM employees
WHERE salary > ALL
(
  SELECT salary
  FROM employees
  WHERE department_id = 90
);
```

Subquery Type : Option 1: Scalar Subquery , Option 2: Multi-Row Subquery

Business Logic Review

The revised queries explicitly handle multiple salaries returned from Department 90 and avoid runtime errors.

Query Logic Analysis

1. Appropriate Subquery Type

Query	Correct Subquery Type
Query 1	Scalar Subquery
Query 2	Multi-Row Subquery
Query 3	Multi-Row Subquery
Query 4	Scalar or Multi-Row Subquery

2. Single-Row vs Multi-Row Results

A scalar subquery must return exactly one value.

Examples:

```
SELECT AVG(salary)
FROM employees;
```

A multi-row subquery may return many values.

Examples:

```
SELECT department_id
FROM employees;
```

When multiple values are returned, operators such as IN, ANY, ALL, or EXISTS should be used.

3. Comparison Operators

Operator	Usage
=	Single value
IN	Multiple values
ANY	At least one value
ALL	All values
EXISTS	Checks existence

Using an incorrect operator often causes runtime errors.

4. SQL Execution Order

Oracle processes SQL statements logically in the following order:

```
FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY
```

Understanding this order explains why aggregate functions cannot be directly used inside the WHERE clause.

5. Business Requirement Validation

A query may be syntactically correct but still fail to satisfy business requirements.

Database engineers must verify:

- Correct table relationships.
 - Appropriate subquery type.
 - Proper comparison operators.
 - Meaningful business interpretation.
 - Accurate result sets.
-

Final Reflection

Why Are Subqueries Useful in Enterprise HR Reporting Systems?

Subqueries allow reports to use intermediate calculations such as average salary, highest salary, department statistics, and management analysis without creating temporary tables.

Difference Between JOIN-Based Thinking and Subquery-Based Thinking

JOINS combine rows from related tables into a single result set.

Subqueries use the result of one query as input to another query.

Example:

JOIN:

`Employee → Department`

Subquery:

`Employee Salary > Company Average Salary`

Why Do Some Subqueries Produce Runtime Errors?

Common reasons include:

- Returning multiple rows when one row is expected.
 - Using the wrong comparison operator.
 - Comparing unrelated columns.
 - Misusing aggregate functions.
-

Why Is SQL Execution Order Important?

SQL clauses are not executed in the order they are written.

Understanding execution order helps prevent logical errors and explains many Oracle runtime errors.

Why Is Understanding Query Logic More Important Than Memorizing Syntax?

Syntax only tells the database how to execute a query.

Business logic determines whether the query solves the intended problem correctly.

A database engineer must understand relationships, data meaning, execution flow, and reporting requirements to produce reliable enterprise reports.